

UNIVERSITY OF BIRMINGHAM

The University of Birmingham

School of Computer Science

Assignment 2 – Simulation System
Deadline: 16:00, Dec 11, 2025

Author:
Pieter Joubert

Reviewers:
Jon Rowe
Ahmad Ibrahim
Wendy Yanez
Sergey Goncharaov

Version 1.0

An Assignment submitted for the UoB:

Object Oriented Programming

November 25, 2025

Contents

1 Process For Assignment 2 **3**

2 Introduction **4**

2.1 Example Simulation Flow 5

3 Mark allocations **5**

3.1 Minimum expected commands 5

3.2 Minimum simulation requirements 6

4 Full class hierarchy **7**

5 Creating the required classes and methods **7**

6 Task 1 - `org.uob.a2.parser` **8**

6.1 Required Methods 8

7 Task 1 - `org.uob.a2.engine` **8**

7.1 Required Methods 8

8 Task 2 - `org.uob.a2.model` **9**

9 Task 5 - `Main.java` **9**

10 Task 6 - New features **9**

11 Submission Procedure **9**

12 Rubric **10**

Rules

1. For each class refer to its corresponding test to verify field and method naming conventions.

2. Although there are many ways to construct an application, you are required to adhere to the rules stipulated below (to achieve marks).

3. If variable names are not stipulated, you can use your own names for variables. This shows that you have written the application (we will check for plagiarism).

4. For Assignment 2 you MAY import additional Java packages in your project, but only those that are part of the standard Java library (e.g., java.util, java.io, etc.). Using third-party libraries or packages not included with the standard JDK is NOT permitted.

5. Do NOT change or modify files included in the "test", "lib" or "out" folders.

6. Do NOT modify the template code. However, you are allowed to create your own methods or classes if they are needed.

7. You MUST complete this assignment independently – Do NOT discuss or share your code with others, and Do NOT use ChatGPT! Any cheating behaviour will result in a zero score for this module and will be subject to punishment by the University.

8. It is *STRONGLY ADVISED AGAINST* utilizing any translation software (such as Google Translate) for the translation of this document.

9. The jUnit tests included in the skeleton code are basic and only scratch the surface in evaluating your code. Passing these tests does not guarantee a full mark.

10. Wrong file structure leads to a substantial penalty. Make sure you have followed the Submission Instructions on the Assignment 2 Canvas page.

11. Make sure you have pushed the version of your code that you want marked to gitlab.

12. Make sure you keep your video recording to the time limit. Any videos longer than 7 minutes will be penalised and videos over 10 minutes will receive zero. DO NOT compress your video in SIZE or TIME.

13. Late submissions are accepted only for 24 hours after the deadline and are penalised at 5% of the final assessment mark. After that 24 hour grace period the submission space is closed and no more submissions are accepted.

IMPORTANT READ THIS FIRST:

1 Process For Assignment 2

Make sure you follow the process as outlined below. Assignment 2 has a different structure and a somewhat different approach to Assignment 1:

1. Clone the repository for **assignment 2**. Make sure you clone **your** repository. (If you haven't yet accessed `gitlab.cs.bham.ac.uk` do so ASAP.)
2. Complete each class in the `src` folder as defined in the assignment.
3. The suggested order for completing classes is:
   (a) All classes in `org.uob.a2.parser`.
   (b) All classes in `org.uob.a2.engine`.
   (c) All classes in `org.uob.a2.model`.
   (d) The `Main.java` class.
4. For each class:
   (a) Complete the required code (methods and attributes).
   (b) Run your code using the `run.sh` command, ensuring it compiles without errors.
   (c) Test your code using the `test.sh` command. You will have errors at first but they should hopefully decrease as you progress.
   (d) `git add`, `git commit`, and `git push` your code.
5. Keeping testing after each class is implemented until everything is working.
6. Add the required new features:
   (a) Extend the `Producer`, `Converter` and `Consumer` classes with new classes. (You can either use the example given or create your own)
   (b) Display a graph of a resource over time.
   (c) Save the current state of a simulation to a text file.
   (d) Load the current state of a simulation from a text file.

2 Introduction

In this assignment, you are tasked with creating a Simulation Engine. This program will be text based like assignment one but will be more complex. The user will use various commands (e.g. "build mine", "info resources") to control the simulation, and output will be displayed as text to the user.

The simulation works as follows:

- The user can build `Producers`, `Converters` or `Consumers`.
- These entities will either produce a resource, convert once resource to another, or consume a resource.
- These entities will cost one or more resources to build.
- You cannot build an entity if you do not enough of that resource.
- Each simulation tick or turn, all producers will produce new resources of a specific type, each converter will try and convert one resource into another and each consumer will try and consume a specific resource.
- A consumer consumes a specific resource and changes its internal state. It does not produce a new resource.
- You need a special final action to do something with the consumers as an end goal for the simulation. (e.g. Selling the final product, Launching the refined metal into spec)
- For example: a `Mine` producer could produce `ORE`.
- A `Refinery` converter could convert `ORE` and `COAOL` into `METAL`.
- A `SpacePort` consumer could consume `METAL` and PEOPLE to update its internal `level`.
- Once full a `SpacePort` could launch the `METAL` into space and provide a big influx of `CREDITS`
- A simulation can have any amount of producers, converters and consumers.
- A simulation can also have any number of types of producers, converters consumers.

IMPORTANT: You can use the example classes we have given (Mine, Forest, Lake, House, Furnace, Refinery, SpacePort) and ResourceTypes (CREDIT, ORE, METAL, WATER, WOOD, COAL, PEOPLE), or you can create your own. If you create your own you will get the last 10 marks on the rubric. If you use our example classes you will not.

2.1 Example Simulation Flow

The following is the simulation flow that you can use if you do not create your own entities and resources (the amount needed and produced is given in brackets):

```
MINE --> ORE(10)
FOREST --> WOOD(10)
LAKE --> WATER(10)
HOUSE --> PEOPLE(1)
WOOD(10) + WATER(10) --> FURNCE --> COAL(1)
ORE(10) + COAL(10) --> REFINERY --> METAL(1)
METAL(10*level) + PEOPLE(10*level) --> SPACEPORT (+1 level)
```

3 Mark allocations

You will receive marks based on two aspects of the game: Firstly, the results of running the **test.sh** command. We will run our own version of these tests once you have submitted. This command will test each class and method (given Tasks 1 - 6). You need to implement **all the classes and methods** as described in this assignment as well as any required for your new features.

Secondly, you will need to submit a screen recording showing you running the simulation and discussing the code. Your screen recording needs to have the following:

- Show your face and your student card.
- Run the simulation showing each of the required commands and any new features your created. (HINT: Use the `cheat` command to speed up this process)
- Show and briefly explain the code in your **Main.java** file.
- Show and briefly explain your **required** new features (save, load, graph).
- Show and briefly explain anything else that you did beyond the normal scope of the assignment. This includes any **additional** classes and methods you've added.

The screen recording **must** be shorter than **7 minutes**. Do **NOT** speed up the recording in any way. You can use a text-to-speech app if you do not want to record your own voice.

3.1 Minimum expected commands

The following is a **complete** list of commands the simulation must be able to parse (values in angle brackets refer to arguments given to a command, the pipe symbol refers to different options):

- `build|b <entity name>`: create a new entity based on its type (e.g., 'build mine'). The newly created entity must have a unique name.

- `info|i <resources|entities>`: Display information about the current resources or entities in the simulation.
- `graph|g <resource>`: Display a text-based graph of the specified resource over time.
- `save|s <filename>`: Save the current current state of the simulation to a text file.
- `load|l <filename>`: Load the current current state of the simulation from a text file.
- `tick|t`: Advance the simulation one tick.
- `help <topic>`: Display this help information or get help on a specific topic (e.g., 'help build' or 'graph').
- A command to perform the final action on your `Consumer`. You can decide on the command name yourself. (e.g. in the example we have given it would be `launch <SpacePort name>` to launch the contents of a `SpacePort` and obtain `CREDITS`)
- `cheat`: A cheat code option that creates a large amount of each resource for testing.
- `quit`: Exit the game.

NOTE: Some of these commands are not included in the template code and need to be added i.e. graph, save, load

3.2 Minimum simulation requirements

The following are the **minimum** requirements for the simulation. You are welcome to add more if you want to:

- At least **four (4)** producers.
- At least **two (2)** converters.
- At least **one (1)** consumer.
- At least **six (6)** resources.

4 Full class hierarchy

```
org.uob.a2
- Main

org.uob.a2.parser
- Command (abstract)
  - BuildCommand (extends Command)
  - CheatCommand (extends Command)
  - HelpCommand (extends Command)
  - InfoCommand (extends Command)
  - InvalidCommand (extends Command)
  - LaunchCommand (extends Command)
  - QuitCommand (extends Command)
  - TickCommand (extends Command)
  - GraphCommand (extends Command)
  - SaveCommand (extends Command)
  - LoadCommand (extends Command)
- Parser

org.uob.a2.engine
- Entity (abstract)
  - Producer (extends Entity implements Tickable)
  - Converter (extends Entity implements Tickable)
  - Consumer (extends Entity implements Tickable)
- Context
- Engine
- SimulationState
- Tickable (interface)

org.uob.a2.model
 - Mine (extends Producer implements Tickable)
 - Forest (extends Producer implements Tickable)
 - Lake (extends Producer implements Tickable)
 - House (extends Producer implements Tickable)
 - Refinery(extends Converter implements Tickable)
 - Furnace (extends Converter implements Tickable)
 - SpacePort (extends Consumer implements Tickable)
 ((Replace these Classes with your own if you are doing your own
     simulation))
- ResourceType (enum)
```

5 Creating the required classes and methods

The required classes and methods fall into three categories: (1) Classes and Methods we have already given you in the template, (2), Classes and Methods you have to create based on inheritance and interfaces, and (3), Classes and Methods we explicitly ask for in the next sections of the assignment.

Please make sure you add ALL of these classes and methods. You are also allowed to add any new Classes or Methods you think will help you complete the assignment.

6 Task 1 - `org.uob.a2.parser`

These classes convert the player's text into `Commands` that can be executed by the game. It firstly sanitises the players input to get it into a correct format. Then it calls the `execute` method on that command, and passes through a `Context`

6.1 Required Methods

- `public Command parse(String command)`

7 Task 1 - `org.uob.a2.engine`

These classes represent the main engine of simulation. It includes the base classes for `Producer`, `Converter` and `Consumer`, as well as the `Tickable` interface. It also contains the `Engine` class that runs the simulation and the `SimulationState` class that stores the current state of the simulation. The `Context` class is a utility class to combine the engine and state into one class.

7.1 Required Methods

- Engine: `public Engine(SimulationState state)` - make sure to add 1000 CREDITS so that the user can buy some initial entities.
- Engine: `public int getCurrentTick()`
- Engine: `public String nextTick()`
- Engine: `public String save(String filename)`
- Engine: `public String load(String filename)`
- Engine: `public String initialiseDefaults()` - this methods creates one of each type of entity for testing.
- SimulationState: `public void addResource(ResourceType resource, int amount)`
- SimulationState: `public void updateResource(ResourceType resource, int amount)`
- SimulationState: `public int getResourceAmount(ResourceType resource`
- SimulationState: `public boolean removeResource(ResourceType resource, int amount)`

- SimulationState: `public void updateHistory()`

8 Task 2 - `org.uob.a2.model`

These classes represent all the concrete entities that will be built in the simulation. You can either use the given examples, or create your own, simulating whatever you want (e.g. a cooking simulation, a farm simulation, a business simulation). The `ResourceType` enum is used to define all possible resources in your simulation.

9 Task 5 - `Main.java`

This is the main java class that parses user input and executes Commands.

10 Task 6 - New features

Displaying a graph of any specified resource. Saving a SimulationState to a file. Loading a SimulationState from a file.

You may add methods and classes as required to get these features to work.

11 Submission Procedure

The general steps to take to complete the project are as follows:

- If you haven't yet done this: Set up your gitlab ssh access using the **setup-git** command on vlab. If you already have a key please do not set it up again.
- Copy your ssh key to your gitlab profile. Once again only do this if you haven't done it for assignment 1.
- Clone the template repository **for assignment 2** from your gitlab. Make sure to clone **YOUR** repository and not the template repository.
- Work on your code, testing it regularly. Use the **run.sh** script to run the code as this builds the code correctly as well.
- Use the **test.sh** script to test your code. This will give you an output similar to what we will use to mark the code.
- Make sure you **commit** and **push** regularly as well.
- Make sure to add **comments** to your code to cleary explain what each method is doing.

- Once you have completed the code, record a short video using any tool you prefer. Refer to Section 3 for more information.
- Submit the video to canvas.
- Submit the latest **commit hash** to canvas.
- If there are no problems you are done with the assignment.
- If there are problems with your submission, update it accordingly and resubmit the latest **commit hash**.

12 Rubric

Task	Submission Type	Mark
Automark Tests Passing	gitlab	30
Simulation Commands Shown	Canvas video submission	20
Main.java	Canvas video submission	10
New features (graph, save, load)	Canvas video submission	30
Creating your own concrete classes	Canvas video submission	10
Total		**100**

Table 1: Mark Rubric